

Coding & Solution

Date	05 July 2026
Team ID	
Project Name	Smart Lender
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/kvnvk9955-stack/Smart-Lender
Programming Language(s)	Python, HTML5, CSS3, JavaScript
Framework(s) Used	Flask, SQLAlchemy, SQLite
Key Features Implemented	• Interactive 3-step form wizard with client-side validation. • Automated preprocessing pipeline (MinMax scaling/Label encoding). • Real-time model inference using a tuned XGBoost Classifier. • SQLite database tracking (Log records for user, profile, loan details). • History logs view dashboard with cascade-delete options.
Pending / Incomplete Features	None
Setup / Run Instructions	1. Install dependencies: pip install flask pandas numpy scikit-learn xgboost flask_sqlalchemy 2. Run the main server application: python app.py 3. Open your browser and navigate to http://localhost:5001

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	YES
2	Meaningful variable and function names are used	YES
3	Code includes comments / documentation where necessary	YES
4	Error handling is implemented for critical operations	YES
5	The application runs without critical errors	YES
6	Code is committed to a version control repository	YES

Additional Notes / Comments:

The codebase is fully organized according to PEP 8 guidelines. The directory structure separates static layout assets, Jinja2 template views, database models, and the core routing controller, making the solution highly maintainable and ready for future updates.